



The Full SQD Pipeline

Lab 3 of 3 · the whole workflow, from molecule to energy

James Weaver · Qiskit Fall Fest 2026

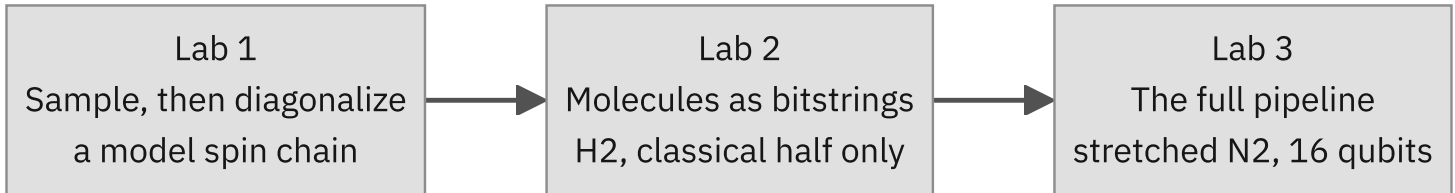
Background

In **May 2016**, IBM put a 5-qubit quantum processor on the cloud, the first one anybody could use from a browser.

A decade later, this session puts the whole method together: the same workflow used in current quantum chemistry calculations, start to finish, at a size that runs on a laptop.

This is not a simplified version of the method. It is the method, run on a system small enough that you can check the answer exactly.

Where this lab fits

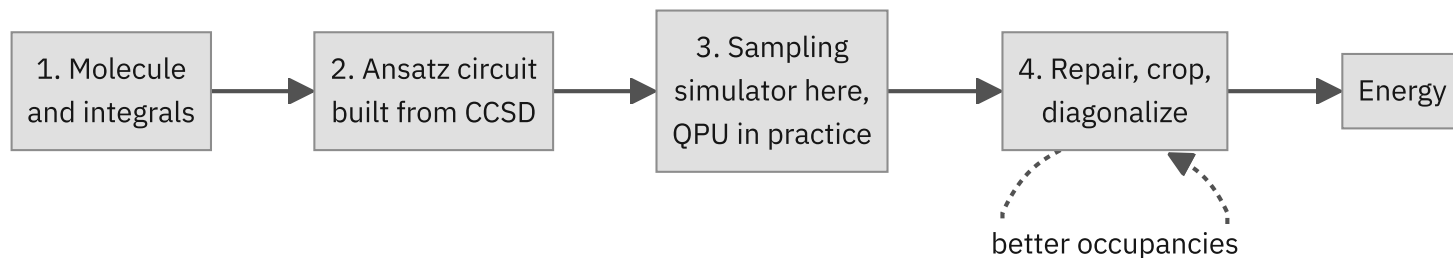


From Lab 1: ground states concentrate on a small number of bitstrings, so you can cut the Hamiltonian down to those and diagonalize the small block. The answer you get is never below the true energy.

From Lab 2: an electron configuration is just a bitstring, and a molecule's Hamiltonian comes from its integrals.

Today those two halves come together, with a real quantum circuit doing the proposing.

The pipeline



Four stages, plus one feedback loop that is what separates this from a single pass.

A few of those labels are terms you have not met yet: *ansatz*, *CCSD*, *configuration recovery*. Each one gets defined in its own section of the notebook, so nothing on this map stays unexplained.

The system: nitrogen, stretched

Nitrogen gas is two nitrogen atoms held together by a triple bond, the strongest bond in everyday chemistry. The notebook looks at it with the two atoms pulled further apart than normal, partway toward breaking.

That is not an artificial setup. Molecules stretch as they vibrate, and much more as they react.

Lab 2 showed what stretching does: several configurations become important at once, and methods built on a single configuration start to fail.

So this system was chosen because classical shortcuts handle it badly, not because it makes the method look good.

Shrinking the problem: the active space

Two moves make a 14-electron molecule fit on a laptop:

- a **smaller basis set**, which means fewer orbitals to track,
- **freezing** the innermost orbitals. Those electrons stay tucked against the nuclei and barely participate, so we lock them in place and stop simulating them.

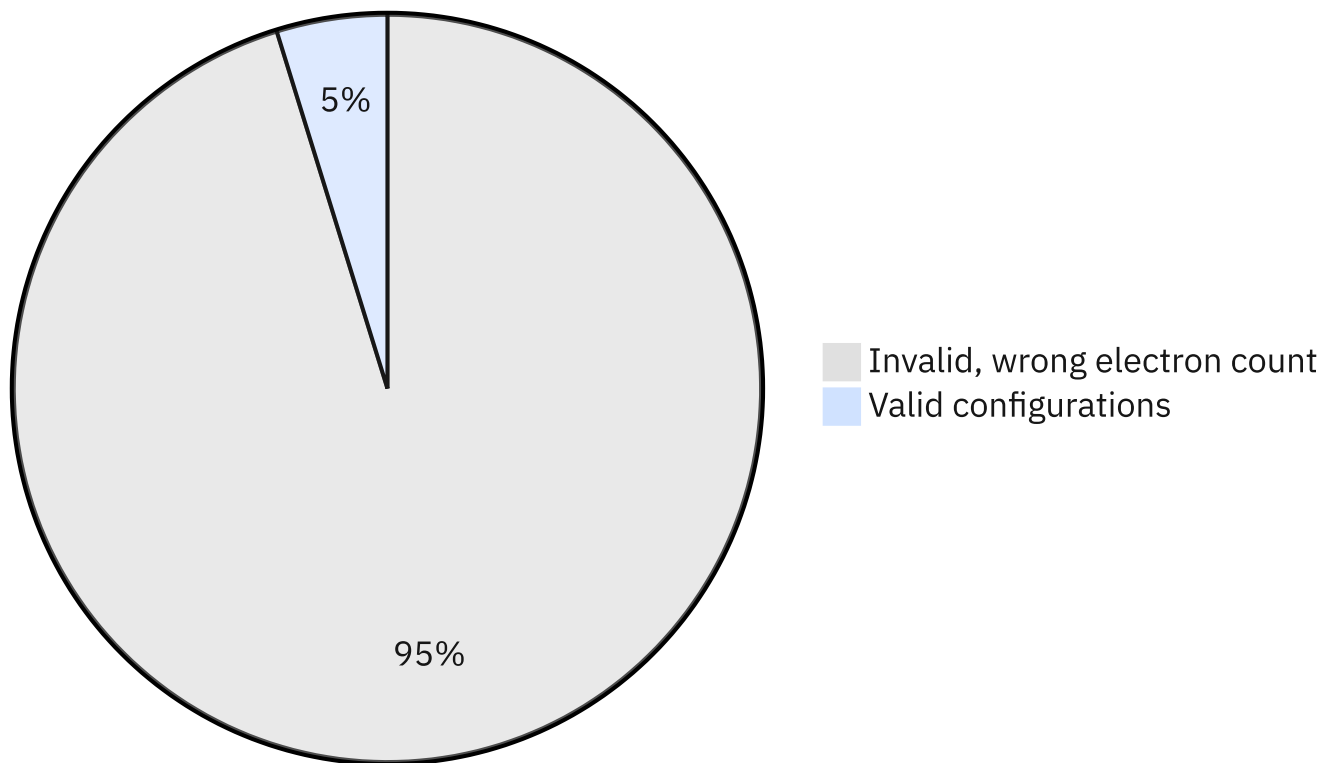
What is left is called the **active space**. The notebook works the arithmetic out step by step, and it lands on **16 qubits**.

Each extra active orbital costs two more simulated qubits, since every orbital has an alpha seat and a beta seat. A laptop simulator gets impractical somewhere around 24.

Most bitstrings are not valid configurations

Sixteen qubits give $2^{16} = 65,536$ possible bitstrings. But a valid configuration has to have exactly the right number of alpha and beta electrons.

Bitstrings on 16 qubits



What follows from that 5%

Under **5%** of bitstrings mean anything physically, and two consequences follow. Between them they organize the rest of the notebook:

- a good circuit should put its probability **only** inside that 5%,
- noise flips bits without caring about electron count, so it keeps knocking samples **out** of it.

The first shapes how the circuit gets built. The second is why the pipeline needs a repair step at all.

The circuit, wired by a classical calculation

An **ansatz** is a circuit whose job is to prepare a rough first draft of the state you want. As Lab 1 showed, it does not have to be very good.

This one is built from a classical chemistry calculation called **CCSD**, which works out which electron moves matter most. Those numbers become the circuit's rotation angles directly.

There is no training loop and no optimizer. That surprises people, because most introductions to quantum algorithms teach the opposite. Nothing in this lab tunes circuit parameters.

The circuit is also built so it **cannot change the electron count**, which means every clean shot lands inside the valid 5% from the previous slide.

What noise does to the samples

The notebook uses a deliberately simple error model: after sampling, flip each recorded bit with some small probability. It is a stand-in for readout error, which is what happens when the hardware misreads a qubit, and the notebook is clear that it does not capture everything a real processor does.

Broken electron counts are the problem. A bitstring with the wrong number of electrons does not describe our molecule at all.

The obvious fix is to throw those samples out. But that discards partly correct information and shrinks your usable sample budget, and it gets worse as systems get bigger.

Configuration recovery: repair instead of discard

Taken together, the valid samples tell you the average occupancy of each orbital: how often each seat is filled. **Configuration recovery** uses that to *fix* the broken samples rather than drop them, flipping the least likely bits until the electron counts come out right.

Then it repeats: diagonalize in the repaired subspace, read off better occupancy estimates, repair again.



Samples that came back with the wrong electron count become valid configurations that contribute to the answer.

This is the part that lets the method work on noisy hardware, and it is why the pipeline diagram has that feedback arrow.

Changing one thing at a time

The last experiment runs four conditions that differ in exactly two ways: clean samples versus noisy ones, and one recovery pass versus several. Everything else is held fixed, including the random seed.

That design is deliberate, and it is part of what the lab is teaching:

Changing one thing at a time is what lets you say *which* change caused *which* difference. Comparing two runs that differ in several ways does not.

What you will do

1. Turn stretched nitrogen into an active-space problem and work out the qubit count.
2. Build the ansatz circuit from a classical CCSD calculation, with no optimizer anywhere.
3. Sample the circuit, then deliberately damage the samples with the bit-flip model.
4. Run configuration recovery and watch broken samples get repaired into usable ones.
5. Run the four-condition comparison that separates noise from recovery depth.
6. **Capstone:** run the whole pipeline on a molecule of your own choosing, with one function call.

🕒 **About 75 minutes**, the longest of the three sessions.

How the notebook works

Run the cells in order. You will see two markers throughout.

- 🧠 **Checkpoints** ask a question and hide the answer in a drop-down. Try to answer before you open it; that is where most of the learning happens.
- ✍️ **Your turn** cells ask you to write code between the `YOUR CODE HERE` markers. A worked solution is in a drop-down right underneath.

If you have not filled in an exercise yet, the notebook stops with a message containing a ✍️. That is the notebook reminding you, not something broken.

Three ways to run this lab

You get a single `.ipynb` file, and it is fully self-contained. There is no `requirements.txt` to fetch, no repository to clone, and no other files it depends on.

OPTION	WHERE IT RUNS	WHAT YOU INSTALL
Your own machine	Local Jupyter	Uncomment the <code>%pip install</code> line in the first cell
Google Colab	Browser, Google account	Uncomment the <code>%pip install</code> line in the first cell
qBraid Lab	Browser, free account	Build one environment, then leave that line commented

You do not need an IBM Quantum account. Everything runs on a local simulator, so it uses no quantum processor time and no credits.

Options 1 and 2 · your machine, or Colab

On your own machine

```
python3 -m venv .venv
source .venv/bin/activate
pip install jupyterlab
jupyter lab
```

Then open the notebook and uncomment the `%pip install ...` line in its first cell, once.

- You need Python **3.11 to 3.14** on macOS or Linux.
- **On Windows, use WSL, Colab, or qBraid.** The chemistry package `pyscf` has no native Windows build.

On Google Colab

Upload the `.ipynb` file, then uncomment and run that same first-cell line. Nothing else changes.

Option 3 · qBraid Lab

A JupyterLab environment that runs in your browser, with a free tier. You build the environment once and reuse it.

1. Sign in at lab.qbraid.com (<https://lab.qbraid.com>).
2. Drag the `.ipynb` file into the file browser.
3. Open the **Environments** panel (upper right), then **Add** → **Create Environment**. Name it `fall-fest-sqd` and paste in this package list:

```
numpy>=2.3.2
qiskit==2.5.1
qiskit-aer==0.17.2
qiskit-addon-sqd==0.12.1
ffsim>=0.0.81
pyscf==2.14.0
matplotlib==3.11.1
pylatexenc==2.10
```

4. When **Installing...** turns into **Add kernel**, click **Add kernel**.
5. Open the notebook, pick the `fall-fest-sqd` kernel in the upper right, and leave the `%pip install` line commented. The environment already has everything.

Getting started

Open `03_the_full_sqd_pipeline.ipynb` and run the first cell.

- One cell takes about twenty seconds. That is not a hang; it is a real 16-qubit simulation.
- When you start experimenting, change one thing at a time.
- Leave time for the capstone. Running the pipeline on your own molecule is the part you take with you.

What you run today follows the standard four-step Qiskit pattern: map the problem, optimize the circuit for the hardware, execute it, then post-process the results.